

Improving Long-Context Preference Following in LLMs via a Trained Preference-Detection Classifier and Enhanced Retrieval

Krishna Jhanwar, 12341250

I. PROJECT DETAILS

This project is undertaken individually. It follows the criterion: **code is available for the base paper (PrefEval), so this project addresses a limitation of the existing methods and proposes an improved solution**, evaluated on the paper’s own benchmark and metrics.

Base Paper: Zhao et al., “Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs,” ICLR 2025 (Oral).

- Paper: [arXiv:2502.09597](#)
- Code: [GitHub – PrefEval](#)
- Dataset (also bundled in the GitHub repo’s `benchmark_dataset` directory as JSON): [PrefEval Explicit](#), [PrefEval Implicit Choice](#), [PrefEval Implicit Persona](#)
- Project page: [PrefEval Project Page](#)

Individual Team:

- Krishna Jhanwar, 12341250, krishnaj@iitbhilai.ac.in

II. PROBLEM STATEMENT

Large Language Models are increasingly deployed as long-running conversational assistants, yet PrefEval [1] shows that state-of-the-art LLMs fail to consistently follow user preferences stated earlier in a conversation, with zero-shot preference-following accuracy falling below 10% at just 10 conversational turns. Even the paper’s strongest baseline methods – Reminder prompting and Retrieval-Augmented Generation (RAG) with a 2021-era SimCSE sentence encoder – still degrade substantially as context length grows toward 100k tokens.

Motivation: Proactive, reliable preference following is central to any real personalized assistant (recommendations, scheduling, tutoring, customer support). The gap identified by PrefEval is therefore both an important open problem in LLM personalization and a well-scoped, measurable one, since the benchmark already provides ground-truth preference-query pairs and an automatic scoring pipeline.

Scope of this project: Given that this is an individual, time-bounded course project, the work is scoped to:

- A subset of topics (to be finalized, e.g. 5–8 of the 20) spanning at least two preference forms (explicit and implicit-choice-based).
- Two representative context-length regimes (short: ~ 10 turns, long: ~ 70 – 100 turns).

- One proprietary LLM and two to three open-source LLMs, chosen based on available API/compute budget (detailed in Section IV).

III. METHODOLOGY

This project addresses two of PrefEval’s own stated weak points – *Long-Context Retrieval* and *Personalization Proactiveness* – by combining a trained classifier with an improved retrieval-and-memory pipeline. The work is carried out in three sequential phases.

Phase 1: Codebase Setup and Baseline Verification

The official PrefEval repository is set up locally, and the existing evaluation scripts are run as-is on a small subset of topics and models to reproduce a slice of the paper’s reported baseline numbers (Zero-shot, Reminder, and RAG, on the classification task). This phase validates that the evaluation pipeline is understood and working correctly before any new component is built, and doubles as the practical on-ramp to the retrieval, prompting, and evaluation concepts the rest of the project depends on.

Phase 2: Preference-Detection Classifier Training

A supervised binary classifier is trained to answer: “does this conversational turn state a user preference?” Training labels are bootstrapped from PrefEval’s own annotated preference turns, supplemented with LLM-generated pseudo-labels on additional conversational data for a larger training set. Candidate base models for this classifier are listed in Section IV. Training/validation loss curves, held-out test accuracy, precision/recall per preference form (explicit, choice-based, persona-driven), and a confusion matrix are recorded as outputs of this phase.

Phase 3: Full Pipeline Evaluation and Comparison

The trained classifier from Phase 2 is integrated with an improved embedding encoder (replacing PrefEval’s 2021 SimCSE retriever) into a structured preference-memory pipeline (Figure 2): the classifier flags preference-stating turns, which are appended to a persistent memory list; at query time, the memory list and top- k retrieved exchanges (using the improved encoder) are combined into the prompt. This full pipeline is then evaluated against PrefEval’s original baselines (Zero-shot, Reminder, Self-Critic, Few-Shot CoT, RAG) using the paper’s

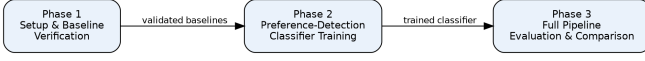


Fig. 1. Three-phase project plan: setup and baseline verification, classifier training, and full pipeline evaluation.

own classification-task accuracy and generation-task LLM-as-judge accuracy, across the chosen topic subset, context-length regimes, and model set.

Difference from existing methods: PrefEval’s five baselines are all single-pass, context-window-dependent strategies, and none of them involve a component trained specifically for this task. The proposed pipeline decouples *preference storage* from *context length* – the memory list grows with the number of stated preferences, not with the number of conversational turns – and replaces repeated LLM-prompted preference detection with a single cheap classifier forward pass per turn, which is also expected to reduce per-turn latency relative to a prompting-based detection step.

Why retrieval is still needed alongside memory: Memory and retrieval are complementary, not redundant. The classifier will not have perfect recall, so retrieval over raw exchanges acts as a fallback for preferences memory misses. Memory entries are also short, normalized summaries that can lose nuance present in the original turn, which retrieval can recover. Retrieval further covers relevant context that is not itself a stated preference. Finally, as the memory list grows, injecting it whole into every prompt reintroduces the long-context problem; retrieval can instead be applied over the memory list itself, surfacing only what is relevant to the current query.

IV. RESOURCES AND DATASET AVAILABILITY

Dataset: PrefEval, 3,000 manually curated preference-query pairs across 20 topics and 3 preference forms (1,000 examples each for explicit, implicit-choice, and implicit-persona forms), publicly available as three HuggingFace dataset repos – [PrefEval Explicit](#), [PrefEval Implicit Choice](#), [PrefEval Implicit Persona](#) – and is also bundled directly in the official GitHub repository’s `benchmark_dataset` directory as JSON. Multi-session distractor context is drawn from the public LMSYS-Chat-1M dataset ([Link](#)), also freely available. No additional data collection is required for the retrieval/memory evaluation; light bootstrapped/pseudo-labeling is needed only for classifier training data, as described in Phase 2.

Candidate classifier base models (Phase 2), all available on HuggingFace:

- `distilbert/distilbert-base-uncased` ([Link](#))
- `answerdotai/ModernBERT-base` ([Link](#))
- `microsoft/MiniLM-L12-H384-uncased` ([Link](#))
- `TinyLlama/TinyLlama-1.1B-Chat-v1.0` ([Link](#)) (heavier alternative, LoRA-adapted)

A small encoder-only model (DistilBERT/ModernBERT/MiniLM scale) is the primary choice, since binary preference detection is a standard text-classification task that does not require a generative model, and keeps training and inference cost low.

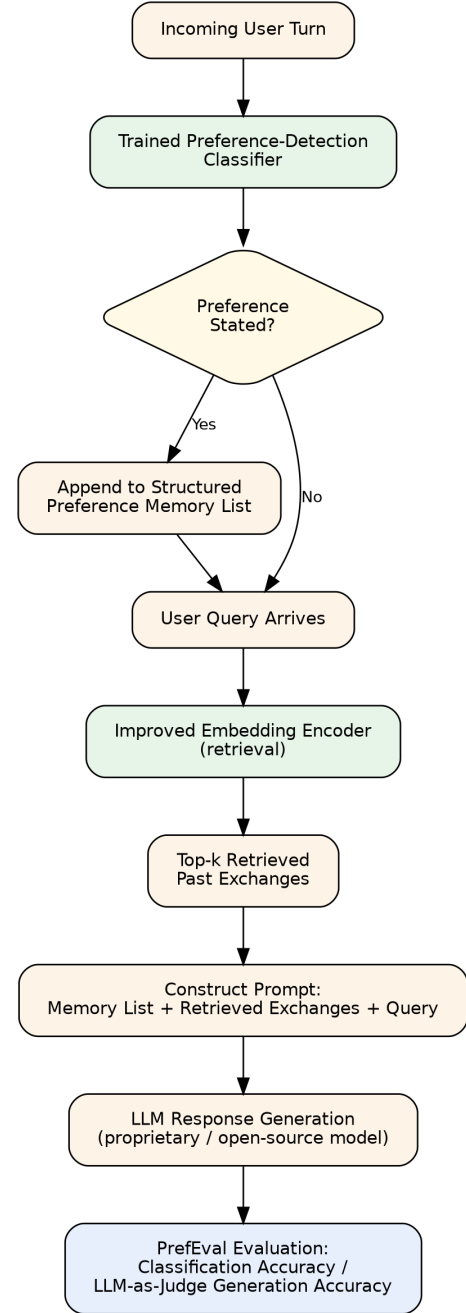


Fig. 2. Proposed pipeline: trained preference-detection classifier feeds a structured memory list, combined with improved-encoder retrieval at query time, then evaluated with PrefEval’s own metrics.

Candidate retrieval embedding models:

- `BAAI/bge-base-en-v1.5` ([Link](#))
- `intfloat/e5-base-v2` ([Link](#))

Proprietary LLM for evaluation: evaluating Claude across the full topic/turn-length grid is likely to be costly for an individually funded project. An OpenAI model (e.g. `gpt-4o-mini`, or `gpt-4o` if budget allows) is planned as the primary proprietary system evaluated, since it offers a lower per-call cost for the volume of calls this evaluation requires.

LLM-as-judge for the generation task: an OpenAI model is planned as the primary judge (for cost reasons); Claude (as used by the original authors) will be used as the judge instead, or as a secondary judge for cross-validation, if budget permits.

Open-source models for evaluation: evaluation is planned over a subset of the open-source model families used in the original paper, such as Llama (e.g. meta-llama/Llama-3.1-8B-Instruct (Link)) and Mistral (e.g. mistralai/Mistral-7B-Instruct-v0.3, (Link)), accessed either locally or via a free/low-cost inference API, to keep the proprietary-API budget focused on the single OpenAI model above.

Hardware: A local CPU/GPU (e.g. free-tier Google Colab GPU) is sufficient. The retrieval-side embedding models are small (base-sized, <500M parameters); the preference-detection classifier is similarly small-scale and trainable on a free-tier GPU in minutes per epoch.

Software: Python, Sci-Kit Learn, PyTorch, HuggingFace transformers, sentence-transformers, and peft (if a LoRA-based classifier is used instead of an encoder model), the PrefEval repository’s evaluation scripts, and API access to the chosen proprietary LLM(s).

Justification: The base dataset and code require no construction effort, allowing the limited individual project timeline to be spent on the classifier and pipeline contribution rather than data collection. Model and API choices above are made explicitly with a single-person budget in mind.

V. LIMITATIONS FOUND

Limitations of the base paper’s methods:

- The RAG baseline uses a dated (2021) sentence encoder, likely under-representing retrieval quality achievable with current embedding models.
- All baselines are reactive at query time; none maintain a persistent, explicitly structured record of previously stated preferences, so recall depends entirely on either the context window or a similarity search over raw dialogue turns.
- In the paper’s own RAG ablation, increasing retrieved context from top-5 to top-10 improves retrieval accuracy but does not improve preference-following accuracy, indicating retrieved context can act as a distractor rather than a help – a gap the original authors note but do not resolve.
- Prompting methods that improve preference adherence (Reminder, CoT) are shown to increase preference *hallucination* in some models, indicating an unaddressed trade-off the base paper does not resolve.

How this project addresses them: The structured preference-memory approach directly targets the second and third points above, using a trained classifier (rather than raw retrieved exchanges) to decide what enters memory, which should reduce the distractor effect observed in the paper’s own ablation. The encoder swap directly targets the first point. Both are evaluated within the base paper’s own metrics for

a fair, comparable result, and hallucination-error rates (one of PrefEval’s four error types) are reported for the proposed method versus baselines to check whether it mitigates or worsens the fourth point.

What differentiates this work: This project does not propose a new benchmark or a new base model; it proposes and empirically validates a lightweight, small-classifier-driven modification to the strongest existing baseline method, reusing the original authors’ own evaluation protocol for a directly comparable result.

VI. EXPECTED OUTCOMES

- Training/validation loss curves, held-out test accuracy, and a precision/recall/confusion-matrix breakdown for the trained preference-detection classifier, reported per preference form.
- PrefEval classification-task accuracy for the full proposed pipeline (trained classifier + memory + improved retrieval) versus the paper’s five baselines (Zero-shot, Reminder, Self-Critic, CoT, RAG), across the chosen topic subset and two context-length regimes.
- A smaller-scale generation-task comparison with LLM-judged accuracy and the four PrefEval error-type breakdown, to sanity-check the classification-task results.
- A plot of accuracy versus context length (turns/tokens), replicating the base paper’s Figure 3 style, showing whether the proposed method degrades more slowly than the baselines.
- A per-turn latency comparison between classifier-based and LLM-prompting-based preference detection.
- Final outputs: a project report/paper documenting the method and results, the evaluation code (as a fork/extension of the official PrefEval repository), and a short slide deck/demo for presentation.

REFERENCES

- [1] S. Zhao, M. Hong, Y. Liu, D. Hazarika, and K. Lin, “Do llms recognize your preferences? evaluating personalized preference following in llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.09597>

APPENDIX

Claude (Anthropic) was used to help draft this proposal document into the required template format, based on the project idea, scope, and methodology described above. The underlying project idea, phase structure, and technical decisions were provided by the author.